

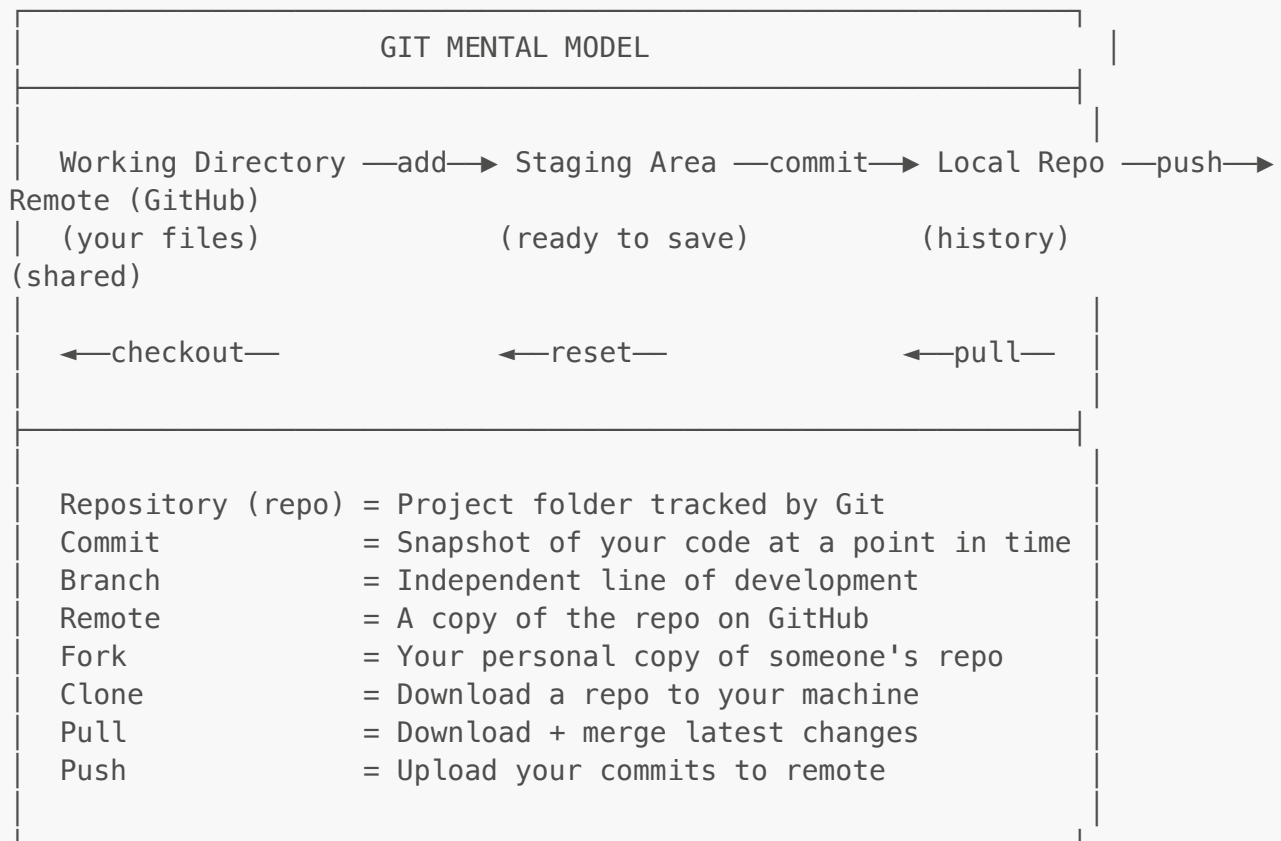
005 — Git Cheatsheet

📦 Git is a version control system that tracks changes to your code. GitHub is the platform where we host our repositories remotely. This cheatsheet covers the workflow you'll use during the bootcamp.

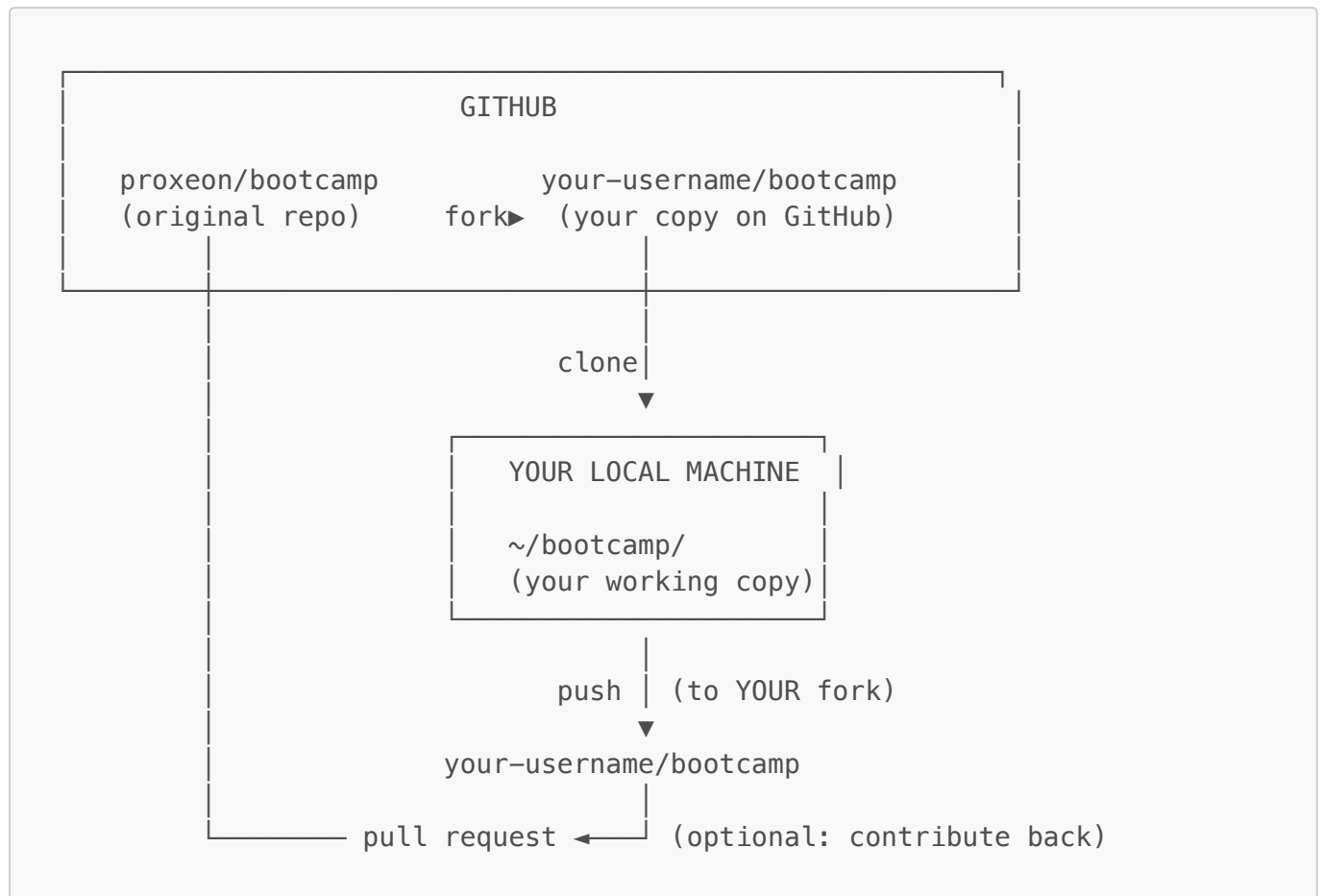
Table of Contents

1. Core Concepts
2. SSH vs HTTPS
3. Bootcamp Workflow: Fork → Clone → Push
4. Essential Git Commands
5. Branching
6. Undoing Mistakes
7. Working with Remotes
8. Git Configuration
9. Common Issues & Fixes
10. Quick Reference Card

1. Core Concepts



How Fork/Clone Relates



2. SSH vs HTTPS

Comparison

Feature	SSH	HTTPS
URL format	<code>git@github.com:user/repo.git</code>	<code>https://github.com/user/repo.git</code>
Authentication	SSH key (automatic)	Username + token (every time)
Setup	One-time key setup	Token management
Recommended	✅ Yes (use this)	Only if SSH is blocked
Firewall-friendly	May be blocked on port 22	Always works (port 443)

How to Tell Which You're Using

```
# Check remote URL
git remote -v

# SSH (✅ what we want):
# origin git@github.com:your-username/bootcamp.git (fetch)
# origin git@github.com:your-username/bootcamp.git (push)
```

```
# HTTPS (❌ switch to SSH):  
# origin https://github.com/your-username/bootcamp.git (fetch)  
# origin https://github.com/your-username/bootcamp.git (push)
```

Switch from HTTPS to SSH

```
git remote set-url origin git@github.com:your-username/bootcamp.git
```

Switch from SSH to HTTPS (fallback)

```
git remote set-url origin https://github.com/your-username/bootcamp.git
```

SSH Over HTTPS Port (If Port 22 is Blocked)

Some networks block port 22. You can tunnel SSH through port 443:

Add to `~/.ssh/config`:

```
Host github.com  
  HostName ssh.github.com  
  Port 443  
  User git  
  IdentityFile ~/.ssh/id_ed25519
```

Test:

```
ssh -T git@github.com
```

3. Bootcamp Workflow: Fork → Clone → Push

Step 1: Fork the Repository

1. Go to: <https://github.com/proxeon/bootcamp>
2. Click the **Fork** button (top-right)
3. Select your personal account as the destination
4. Wait for the fork to complete

You now have https://github.com/YOUR_USERNAME/bootcamp

Step 2: Clone YOUR Fork to Local Machine

```
# Clone using SSH (recommended)
git clone git@github.com:YOUR_USERNAME/bootcamp.git

# Navigate into the project
cd bootcamp
```

Step 3: Add the Original Repo as "Upstream"

This lets you pull updates from the instructor's repo later:

```
git remote add upstream git@github.com:proxeon/bootcamp.git
```

Verify remotes:

```
git remote -v
# origin      git@github.com:YOUR_USERNAME/bootcamp.git (fetch)
# origin      git@github.com:YOUR_USERNAME/bootcamp.git (push)
# upstream    git@github.com:proxeon/bootcamp.git (fetch)
# upstream    git@github.com:proxeon/bootcamp.git (push)
```

Step 4: Make Changes

```
# Edit files...
# For example, create your .env file:
cp .env.example .env
nano .env
```

Step 5: Stage and Commit Changes

```
# See what changed
git status

# Stage specific files
git add .env
git add docker-compose.prod.yml

# Or stage everything
git add .

# Commit with a message
git commit -m "feat: configure environment for deployment"
```

Step 6: Push to YOUR Fork

```
git push origin main
```

Step 7: (Optional) Sync with Upstream

If the instructor pushes updates to the original repo:

```
# Fetch latest from instructor's repo
git fetch upstream

# Merge their changes into your main branch
git merge upstream/main

# Push the merged result to your fork
git push origin main
```

Complete Bootcamp Workflow Diagram

ONE TIME SETUP

1. Fork on GitHub
2. `git clone git@github.com:YOU/bootcamp.git`
3. `git remote add upstream git@github.com:proxeon/...`

DAILY WORKFLOW

1. `git pull origin main` (get your latest)
2. (make changes)
3. `git add .` (stage changes)
4. `git commit -m "message"` (save snapshot)
5. `git push origin main` (upload to GitHub)

→ GitHub Actions auto-triggers → Images built → VPS

SYNC WITH INSTRUCTOR

1. `git fetch upstream`
2. `git merge upstream/main`
3. `git push origin main`

4. Essential Git Commands

4.1 Getting Information

Command	Description
<code>git status</code>	Show modified/staged/untracked files
<code>git log</code>	Show commit history
<code>git log --oneline</code>	Compact commit history
<code>git log --oneline -10</code>	Last 10 commits
<code>git log --graph --oneline</code>	Visual branch graph
<code>git diff</code>	Show unstaged changes
<code>git diff --staged</code>	Show staged changes
<code>git diff HEAD~1</code>	Changes since last commit
<code>git show COMMIT_HASH</code>	Show specific commit details
<code>git blame filename</code>	Who changed each line

4.2 Making Changes

Command	Description
<code>git add file.txt</code>	Stage a specific file
<code>git add .</code>	Stage all changes
<code>git add -p</code>	Interactive staging (chunk by chunk)
<code>git commit -m "message"</code>	Commit with message
<code>git commit -am "message"</code>	Stage tracked files + commit
<code>git commit --amend</code>	Modify the last commit
<code>git commit --amend -m "new msg"</code>	Change last commit message

4.3 Commit Message Convention

```
type: short description
```

```
# Types:
```

```
feat:    New feature
```

```
fix:     Bug fix
```

```
docs:    Documentation only
```

```
style:   Formatting (no code change)
```

```
refactor: Code restructuring
```

```
test:    Adding tests
```

```
chore:    Maintenance (deps, config)
ci:       CI/CD changes
```

Examples:

```
git commit -m "feat: add Docker deployment configuration"
git commit -m "fix: resolve database connection timeout"
git commit -m "docs: update README with setup instructions"
git commit -m "ci: enable automated deployments"
git commit -m "chore: update dependencies"
```

4.4 Syncing with Remote

Command	Description
<code>git push</code>	Push current branch to remote
<code>git push origin main</code>	Push main to origin
<code>git push -u origin main</code>	Push + set upstream tracking
<code>git pull</code>	Fetch + merge from remote
<code>git pull origin main</code>	Pull specific branch
<code>git fetch</code>	Download changes without merging
<code>git fetch --all</code>	Fetch from all remotes

5. Branching

5.1 Branch Commands

Command	Description
<code>git branch</code>	List local branches
<code>git branch -a</code>	List all branches (local + remote)
<code>git branch feature-x</code>	Create new branch
<code>git checkout feature-x</code>	Switch to branch
<code>git checkout -b feature-x</code>	Create + switch (shortcut)
<code>git switch feature-x</code>	Switch (newer syntax)
<code>git switch -c feature-x</code>	Create + switch (newer syntax)
<code>git branch -d feature-x</code>	Delete branch (safe)
<code>git branch -D feature-x</code>	Delete branch (force)

Command	Description
<code>git push origin --delete feature-x</code>	Delete remote branch

5.2 Feature Branch Workflow

```
# 1. Start from latest main
git checkout main
git pull origin main

# 2. Create feature branch
git checkout -b feature/add-watchtower

# 3. Make changes and commit
git add .
git commit -m "feat: add watchtower for auto-updates"

# 4. Push feature branch
git push origin feature/add-watchtower

# 5. Create Pull Request on GitHub (optional)

# 6. After merge, clean up
git checkout main
git pull origin main
git branch -d feature/add-watchtower
```

5.3 Merge vs Rebase (Simple Explanation)

```
# Merge: combines branches (creates merge commit)
git checkout main
git merge feature-x

# Rebase: replays your commits on top of main (cleaner history)
git checkout feature-x
git rebase main
```

For the bootcamp: Just use `git merge` or work directly on `main`. Don't worry about rebase.

6. Undoing Mistakes

6.1 "I haven't committed yet"

```
# Discard changes to a specific file
git checkout -- filename.txt
# Or (newer syntax):
```



```
git restore filename.txt

# Discard ALL uncommitted changes
git checkout -- .
# Or:
git restore .

# Unstage a file (keep changes, remove from staging)
git reset HEAD filename.txt
# Or:
git restore --staged filename.txt

# Unstage everything
git reset HEAD .
```

6.2 "I committed but haven't pushed"

```
# Undo last commit, keep changes staged
git reset --soft HEAD~1

# Undo last commit, keep changes unstaged
git reset HEAD~1

# Undo last commit, discard changes completely ⚠️
git reset --hard HEAD~1

# Undo last 3 commits
git reset --soft HEAD~3

# Change last commit message
git commit --amend -m "corrected message"

# Add forgotten file to last commit
git add forgotten-file.txt
git commit --amend --no-edit
```

6.3 "I already pushed"

```
# Create a NEW commit that undoes the previous one (safe)
git revert HEAD
git push

# Force push to rewrite history (⚠️ use cautiously with shared branches)
git reset --hard HEAD~1
git push --force-with-lease
```

6.4 "I committed to the wrong branch"

```
# Move last commit to a new branch
git branch correct-branch    # Create branch at current position
git reset --hard HEAD~1      # Remove commit from current branch
git checkout correct-branch  # Switch to correct branch
```

6.5 "I need to temporarily save work"

```
# Stash uncommitted changes
git stash

# Stash with a description
git stash push -m "WIP: database config"

# List stashes
git stash list

# Apply most recent stash (keep in stash list)
git stash apply

# Apply and remove from stash list
git stash pop

# Apply specific stash
git stash apply stash@{2}

# Drop a stash
git stash drop stash@{0}

# Clear all stashes
git stash clear
```

7. Working with Remotes

7.1 Remote Management

Command	Description
<code>git remote -v</code>	List remotes with URLs
<code>git remote add NAME URL</code>	Add new remote
<code>git remote remove NAME</code>	Remove remote
<code>git remote rename old new</code>	Rename remote
<code>git remote set-url origin URL</code>	Change remote URL
<code>git remote show origin</code>	Detailed remote info

7.2 Typical Remote Setup for Bootcamp

```
# After forking and cloning:
git remote -v
# origin      git@github.com:YOUR_USERNAME/bootcamp.git (fetch)
# origin      git@github.com:YOUR_USERNAME/bootcamp.git (push)

# Add instructor's repo as upstream:
git remote add upstream git@github.com:proxeon/bootcamp.git

# Now you have:
# origin      = YOUR fork (you push here)
# upstream    = instructor's repo (you pull from here)
```

7.3 Syncing a Fork

```
# Get latest from instructor
git fetch upstream

# View what's different
git log HEAD..upstream/main --oneline

# Merge instructor's changes into your main
git checkout main
git merge upstream/main

# If there are conflicts, resolve them, then:
git add .
git commit -m "merge: sync with upstream"

# Push to your fork
git push origin main
```

7.4 Force Push (When Needed)

```
# Safe force push (fails if someone else pushed)
git push --force-with-lease

# Dangerous force push (overwrites everything) ⚠️
git push --force
```

⚠️ **Never force push to a shared branch** unless you know what you're doing.

8. Git Configuration

8.1 Essential Setup

```
# Set your identity (REQUIRED)
git config --global user.name "Your Full Name"
git config --global user.email "your_email@example.com"

# Default branch name
git config --global init.defaultBranch main

# Better diff output
git config --global diff.colorMoved zebra

# Auto-correct typos (waits 1.5s before executing)
git config --global help.autocorrect 15
```

8.2 Useful Aliases

```
# Short status
git config --global alias.s "status -sb"

# Pretty log
git config --global alias.lg "log --oneline --graph --decorate -20"

# Unstage shortcut
git config --global alias.unstage "restore --staged"

# Last commit
git config --global alias.last "log -1 HEAD --stat"

# Quick amend (no message edit)
git config --global alias.oops "commit --amend --no-edit"
```

Usage:

```
git s      # Short status
git lg     # Pretty graph log
git unstage . # Unstage all files
git last   # Show last commit
git oops    # Amend last commit silently
```

8.3 View Configuration

```
# Show all config
git config --global --list
```

```
# Show specific value
git config --global user.email

# Show where config is stored
git config --list --show-origin
```

8.4 .gitignore

Files/folders to exclude from version control:

```
# Environment files (contain secrets!)
.env
.env.local
.env.production

# Dependencies
node_modules/
.pnpm-store/

# Build outputs
dist/
.next/
bin/
obj/

# OS files
.DS_Store
Thumbs.db

# IDE
.vscode/
.idea/
*.swp

# Docker
docker-compose.override.yml
```

Check if a file is ignored:

```
git check-ignore -v filename
```

9. Common Issues & Fixes

9.1 "Permission denied (publickey)" on Push/Pull

Cause: SSH key not configured for GitHub.

```
# Test SSH connection
ssh -T git@github.com

# If it fails, check:
ssh-add -l                                # Is key loaded?
cat ~/.ssh/id_ed25519.pub                 # Does it match GitHub?

# Fix: reload key
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
```

Also verify remote URL uses SSH:

```
git remote -v
# Should show: git@github.com:... (NOT https://...)

# Fix if HTTPS:
git remote set-url origin git@github.com:YOUR_USERNAME/bootcamp.git
```

9.2 "fatal: not a git repository"

Cause: You're not inside a git-tracked folder.

```
# Check current directory
pwd

# Navigate to your project
cd ~/bootcamp

# Verify it's a git repo
ls -la .git/
```

9.3 "error: failed to push some refs"

Cause: Remote has changes you don't have locally.

```
# Pull first, then push
git pull origin main
git push origin main

# If pull creates a merge conflict, see section 9.6
```

Or if you want to rebase (cleaner history):

```
git pull --rebase origin main
git push origin main
```

9.4 "fatal: refusing to merge unrelated histories"

Cause: Trying to merge two repos that have no common ancestor.

```
git pull origin main --allow-unrelated-histories
```

9.5 "Your branch is ahead of 'origin/main' by X commits"

Cause: You have local commits that haven't been pushed.

```
# Just push them
git push origin main
```

9.6 Merge Conflicts

What it looks like:

```
<<<<<<< HEAD
your local changes
=====
changes from remote
>>>>>>> origin/main
```

How to resolve:

```
# 1. Open the conflicting file(s)
git status    # Shows which files have conflicts

# 2. Edit the file – keep what you want, remove the markers
#    Remove <<<<<<< HEAD, =====, and >>>>>>> lines

# 3. Stage the resolved file
git add filename.txt

# 4. Complete the merge
git commit -m "merge: resolve conflict in filename.txt"

# 5. Push
git push origin main
```

Abort a merge (go back to before the merge):

```
git merge --abort
```

9.7 "Changes not staged for commit" / Dirty Working Directory

```
# See what's modified
git status

# Option A: Commit the changes
git add .
git commit -m "save current work"

# Option B: Discard the changes
git restore .

# Option C: Stash for later
git stash
```

9.8 Accidentally Committed Secrets / .env File

```
# Remove from git tracking (keep local file)
git rm --cached .env
echo ".env" >> .gitignore
git add .gitignore
git commit -m "fix: remove .env from tracking"
git push

# ⚠️ If you pushed secrets to a public repo:
# 1. Rotate/regenerate ALL exposed credentials immediately
# 2. The secret is in git history forever (even after deletion)
# 3. Consider using: git filter-branch or BFG Repo Cleaner
```

9.9 "detached HEAD state"

Cause: You checked out a specific commit instead of a branch.

```
# Get back to main branch
git checkout main

# If you made changes in detached state, save them:
git stash
git checkout main
git stash pop
```


9.10 Large Files / Slow Push

```
# Check what's taking space
git rev-list --objects --all | git cat-file --batch-check='%(objecttype) %
(objectname) %(objectsize) %(rest)' | sort -k3 -n | tail -20

# If node_modules got committed accidentally:
git rm -r --cached node_modules
echo "node_modules/" >> .gitignore
git add .gitignore
git commit -m "fix: remove node_modules from tracking"
```

9.11 WSL: Line Ending Issues (CRLF vs LF)

Symptom: Git shows every file as modified even though you changed nothing.

```
# Configure Git to handle line endings
git config --global core.autocrlf input      # WSL/macOS
# git config --global core.autocrlf true     # Windows native (not WSL)

# Fix existing repo
git rm --cached -r .
git reset --hard
```

9.12 "RPC failed; HTTP 400 curl 22"

Cause: Push too large (big files or many commits).

```
# Increase Git buffer size
git config --global http.postBuffer 524288000

# Push in smaller batches
git push origin main --force-with-lease
```

10. Quick Reference Card

The 5 Commands You'll Use 90% of the Time

git status	# What changed?
git add .	# Stage everything
git commit -m "message"	# Save snapshot
git push origin main	# Upload to GitHub
git pull origin main	# Download latest

Bootcamp Day — Complete Flow

```
# — INITIAL SETUP (once) —————
# Fork on GitHub UI, then:
git clone git@github.com:YOUR_USERNAME/bootcamp.git
cd bootcamp
git remote add upstream git@github.com:proxeon/bootcamp.git

# — MAKE CHANGES & DEPLOY —————
# Edit your files (.env, docker-compose, etc.)
git add .
git commit -m "feat: configure deployment"
git push origin main
# → GitHub Actions triggers → Images built → VPS auto-updates!

# — GET INSTRUCTOR UPDATES —————
git fetch upstream
git merge upstream/main
git push origin main

# — CHECK WHAT HAPPENED —————
git status           # Current state
git log --oneline -5  # Last 5 commits
git remote -v        # Verify remotes
```

Emergency Commands

```
# "I want to undo everything I just did"
git reset --hard HEAD

# "I want to go back to what's on GitHub"
git fetch origin
git reset --hard origin/main

# "I committed secrets, make it go away"
git reset --soft HEAD~1
git rm --cached .env
git commit -m "remove secrets"
git push --force-with-lease

# "Everything is broken, start fresh"
cd ..
rm -rf bootcamp
git clone git@github.com:YOUR_USERNAME/bootcamp.git
cd bootcamp

# "I want to save my work but switch tasks"
git stash push -m "WIP: my changes"
```

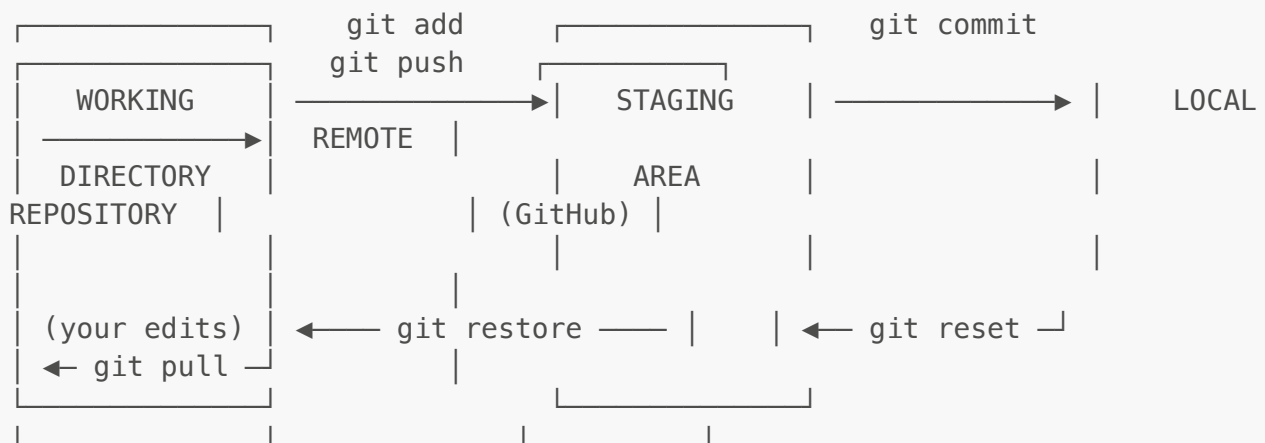
```
# (do other stuff)
git stash pop
```

Status Symbols Explained

```
git status
```

```
# ?? file.txt    → Untracked (new file, not in git)
# M  file.txt    → Modified (changed since last commit)
# A  file.txt    → Added (staged for commit)
# D  file.txt    → Deleted
# R  file.txt    → Renamed
# UU file.txt    → Conflict (needs manual resolution)
```

Visual: Git Stages



CI/CD Trigger: How Push Triggers Deployment

After you `git push origin main`, the following happens automatically:

```
YOU: git push origin main
↓
GITHUB: detects push to main branch
↓
GITHUB ACTIONS: .github/workflows/deploy.yml runs
• Builds Docker images (api, web, admin)
• Pushes to ghcr.io/proxeon/bootcamp/...
```

```
GHCR: new :latest images available
  |
  ▼ (within ~5 minutes)
WATCHTOWER (on VPS): detects new images
  |
  • Pulls new images
  • Stops old containers
  • Starts new containers
  |
  ▼
🎉 YOUR APP IS UPDATED! (zero SSH required)
```

You don't need to SSH into the server. Just push code and wait ~5 minutes.